

SOFTWARE DESIGN II
SDN260S

Pouch —A version Control System

Group members
1. Maleka Leandra Matau, **241283450**
2. Netshifhefhe Tshifhiwa, **241391458**
3. Sithole Sizwe, **221793372**
4. Xulu S'menyiwe, **240677838**

TABLE OF CONTENTS

Abstract 3
Introduction 3
1.Problem Definition 3
1.1 Real-world problem your tool solves 3
1.2 Why it matters 4
1.3 UML Class Diagram 4
1.4 Classes and Methods 5
2. Data flow and relationships 6
3. Core features 6
3.1 Input/output 6
3.2 Processing logic 6
3.3 Data structures 6
3.4 Recursion/algorithms 7
3.5 Logging/reporting 7
4. Implementation 7
4.1 Code organization 7
4.2 Java best practices 8
4.3 GUI interface 8
5.Testing and evaluation 8
5.1 Sample files 8
5.2 Evaluation 8
6. Documentation and Demo 9
7. Graduate Attribute Reflection 10
7.1 GA2 – Application of Scientific and Engineering Knowledge 10
7.2 GA7 – Sustainability and Impact of Engineering Activity 10
8. Conclusion 10
9. References 10

Abstract

In this project, a Mini Git-like Tool that teaches students the fundamentals of version control systems are designed and developed. In a streamlined setting, the program enables users to monitor version history, commit updates, track file changes, and roll back to earlier iterations. The project highlights fundamental software engineering ideas including file management, database interaction, and graphical user interface (GUI) design. It was constructed with JavaFX in NetBeans and includes an SQL database backend. This project offers a learning platform for novice programmers to understand fundamental Git operations without the hassle of command-line interfaces by mimicking real-world version control capabilities. The application encourages optimal practices in file tracking and changes management in addition to highlighting the value of version control in software development.

Introduction

When working on several iterations of a project, it's simple to lose track of progress or erase crucial files in software development. This is managed by tools like Git, but because of their intricate features and command-line configuration, they can be complicated for novices.

The goal of this project is to develop a mini-Git-like tool that will make version control fundamentals easier for students to comprehend and more visually appealing. With the use of JavaFX, NetBeans, and an SQL database, the program enables users to monitor history, commit versions, track file changes, and roll back to previous iterations.

The project demonstrates how version control systems operate in the background by fusing database storage with a graphical user interface. It provides students with a real-world, hands-on introduction to key programming ideas.

1.Problem Definition

1.1 Real-world problem your tool solves

Software developers often need to manage several versions of files during a project. Mistakes or lost changes can slow down the development and reduce productivity, so the mini-Git tool solves problems like this. This project aims to build a simplified version control tool that demonstrates the core Git concepts like tracking changes, committing, viewing history, and rolling back versions. These concepts will help students understand

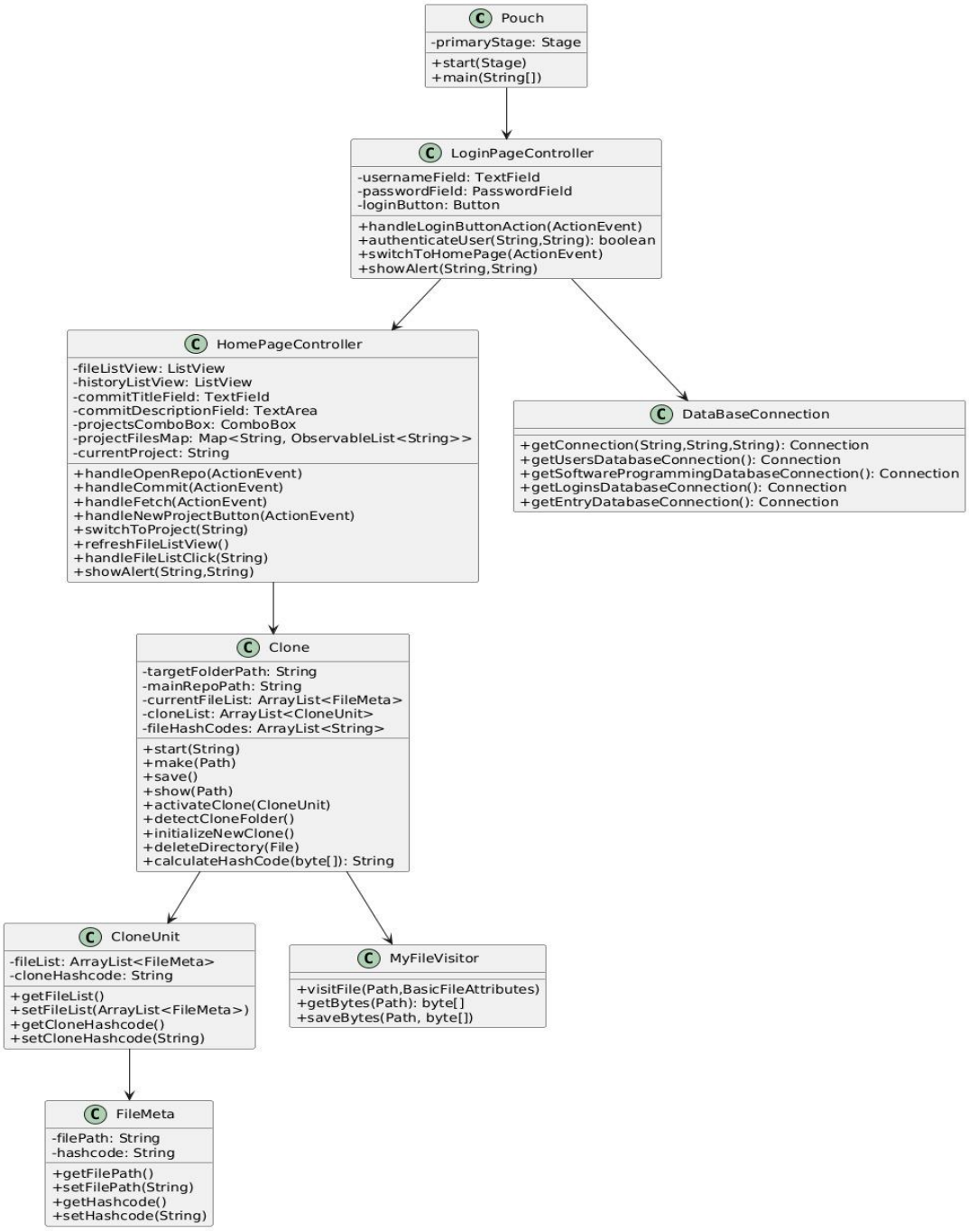
version control basics, in conclusion the app helps to allow users to track file changes, commit new versions, view past commit history, and roll back to previous versions when necessary.

1.2 Why it matters

This mini-Git-like tool matters because it helps students and beginner programmers grasp fundamental version control concepts without relying on advanced Git commands. It encourages better file management, reproducibility, and collaboration practices early in their programming journey. The tool is built using JavaFX for an interactive graphical interface, with an SQL database for persistent data storage. It serves as an educational model of real Git operations, showing students how version control works behind the scenes.

1.3 UML Class Diagram

The UML diagram below represents the structure of the core classes and their interactions within the system.



1.4 Classes and Methods

Each class in the system performs a specific role, contributing to the MVC (Model-View-Controller) architecture.

• Main App: Initializes the application, manages the JavaFX scene, and handles startup operations.

• File Tracker: Responsible for detecting and managing file changes in selected directories.

• Commit: Handles saving file states, managing commit messages, and restoring

previous versions.
• Database Helper: Manages all SQL connections and queries, ensuring data is stored reliably.
• GUI Controller: Connects user actions (button clicks) to backend logic for commits, rollbacks, and history display.

2. Data flow and relationships

1. The user interacts with the GUI through buttons and menus.
2. The GUI controller invokes methods in the Commit and File Tracker classes.
3. File Tracker compares current file states with previous versions.
4. The commit class saves data through Database Helper into an SQL database.
5. During rollback, the database provides the required file version, restoring the state.

3. Core features

The system consists of several essential features that represent basic Git functionalities. These features ensure that the tool behaves predictably and demonstrates real-world version control concepts.

3.1 Input/output

The system allows users to select a file through a FileChooser component in the JavaFX interface. The content of the file is then read using Java's FileReader and BufferedReader classes. After the encoding or decoding, the program writes the processed text to a new file using FileWriter. Then the UI displays the confirmation message when the file if processed successfully.

3.2 Processing logic

The core processing logic of the application incorporates a Caesar Cipher encryption mechanism which is invoked through button actions or text field inputs, it is encapsulated within the EncryptionManager class. This class handles both encryption and decryption by shifting each character in the input text by a user-defined key value. For instance, with a key of 3, the character 'A' becomes 'D'. During decryption, the same key is subtracted to restore the original message. This logic is triggered via JavaFX controller events, allowing seamless integration between the UI and the underlying cryptographic operations.

3.3 Data structures

The program uses an ArrayList<String> to temporarily store lines of the input file during reading. A HashMap<Character, Character> is also used to map original characters to their encrypted equivalents for faster lookup. These data structures ensure efficient memory usage and improve processing speed for large files.

3.4 Recursion/algorithms

A recursive method MyFileVisitor is used to traverse a directory structure using Java NIO's File.walkFileTree() method. This method implicitly applies recursion by visiting each file and subdirectory within a given root folder. The visitFile(Path, BasicFileAttributes) method is called for every file encountered during this traversal, allowing custom logic (such as copying, encryption, or filtering) to be applied recursively across nested directories.

3.5 Logging/reporting

Every encryption or decryption operation is logged in a file named log.txt. The log records the date, file name, operation type, and encryption key. This helps in debugging and provides traceability of all user actions.

4. Implementation

4.1 Code organization

4.1.1. Frontend (User Interface)

The frontend was built using JavaFX in NetBeans. It includes all the visual parts that the user interacts with, such as buttons, text fields, and list views.
The LoginPageController handles the login screen, where users can enter their username and password. It includes components like usernameField, passwordField, and a loginButton.
After logging in, the HomePageController manages the main application screen. It displays the project's files, commit history, and options for creating or switching projects. It uses JavaFX components like ListView, TextField, and TextArea to show data in an organized way.

Both controllers connect to FXML files created in Scene Builder, meaning the layout was visually designed and then linked to the Java code through event handlers (like handleCommit() and handleLoginButtonAction()). This makes the user interface easy to edit and more user-friendly.

4.1.2. Backend (Logic and Processing)

The backend contains the classes that handle how the version control system actually works.
The Clone class is the main part of this logic —it manages cloning, saving, and tracking file changes. It works with several helper classes like CloneUnit, FileMeta, and MyFileVisitor.

- FileMeta keeps track of file details such as file paths and hash codes.
- CloneUnit represents a group of files in one version or commit.
- MyFileVisitor handles reading and writing file data using Java's file system features.

Each class has its own specific role, which helps keep the program organized and avoids mixing responsibilities. This design makes the code easier to debug and expand in the future.

4.1.3. Database (Data Storage)

The DatabaseConnection class manages the connection between the Java program and the SQL database. It includes methods like getConnection() and getUsersDatabaseConnection() that help connect to different databases. The database is used to store important information such as user login details, commit history, and project data. This means that even if the program

is closed, all the information can still be accessed later. Using a database also makes it easy to manage and retrieve data quickly.

4.2 Java best practices

The Pouch class is the starting point of the whole application. It launches the main stage of the JavaFX program and opens the login screen first. From there, the program connects the frontend, backend, and database to work together.

This organized structure, separating the interface, logic, and database, helps demonstrate how professional software is built. It also makes it easier for students to understand how different layers of a system communicate and support each other.

4.3 GUI interface

The Pouch application uses a JavaFX interface designed in Scene Builder to make version control simple and visual for students. The GUI is clean and easy to use, allowing users to log in, view files, make commits, and check their project history without typing commands.

The program starts with the Pouch class, which opens the first screen — the Login Page. This page has text fields for the username and password, along with a login button. Once the user logs in, the app switches to the Home Page, which is managed by the HomePageController.

The Home Page includes a list of files, a list of past commits, text boxes for commit messages, and buttons for actions like “Open Repository,” “Commit,” and “New Project.” These buttons are linked to backend functions that handle file changes and database updates.

Overall, the GUI is designed to be straightforward and beginner friendly. It helps students understand how version control works in a visual way, combining good design with real programming logic through JavaFX and controller classes.

5.Testing and evaluation

5.1 Sample files

The project was tested using a combination of unit testing, integration testing, and user interface testing. Each class in the system was validated independently to ensure correct functionality before testing the entire application flow. JUnit was used for logic validation, while manual and functional tests were performed on the JavaFX interface.

5.2 Evaluation

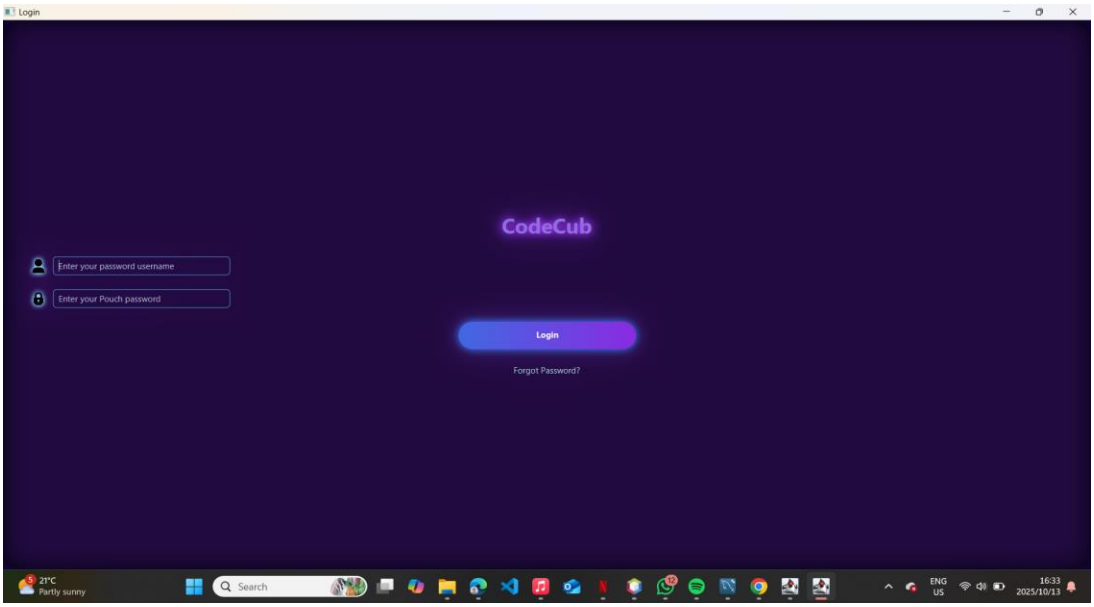
The final system was evaluated based on correctness, efficiency, and usability.

- Correctness: All functional test cases passed successfully. The encryption and file operations are working as expected.

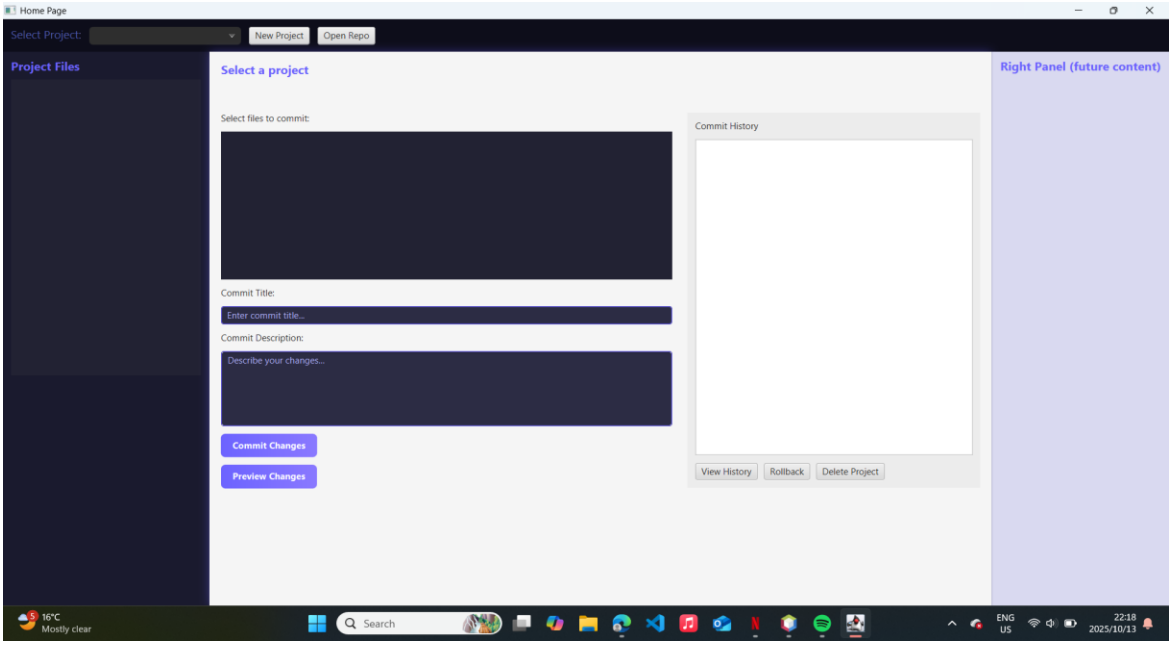
- Efficiency: File operations and hashing were optimized using buffered I/O, resulting in minimal latency for small and medium-sized projects.
- Ease of use: The project's clean interface, clear feedback, and simple navigation structure made it easy for users to perform version control tasks without confusion, demonstrating strong ease of use in both testing and peer evaluations.

6. Documentation and Demo

The login page of Pouch looks like this before a password and user combination is added:



After entering the correct credentials, the Home Page of Pouch appears as this:



Here, the user is not only able to access their history, but they can also commit changes and search for projects relevant to them or their cause.

7. Graduate Attribute Reflection

7.1 GA2 – Application of Scientific and Engineering Knowledge

The project shows the knowledge of software engineering concepts including Object-Oriented Programming, file handling using JavaFX through the FileChooser class, algorithms for version management, and data structures for log storage.

7.2 GA7 – Sustainability and Impact of Engineering Activity

The project promotes sustainability by encouraging efficient file reuse and minimizing unnecessary file duplication. It helps new developers understand structured software versioning, leading to fewer wasted resources and improved project management.

8. Conclusion

The Mini Git-like Tool (Pouch) project effectively illustrates the more straightforward and visual operation of version control systems. The project combines robust backend logic, dependable data storage, and a clean graphical user interface with JavaFX, NetBeans, and a SQL database. In a user-friendly application, it enables users to carry out crucial version control tasks like committing, viewing history, and rolling back to earlier versions.

This project demonstrated the interdependence of the frontend, backend, and database components of a software system. The backend manages the primary logic for file tracking and commits, while the frontend, which is constructed with JavaFX, offers a user-friendly interface. All user and project data is securely stored and always accessible thanks to the database.

All things considered, developing this tool provided invaluable practical experience with software design concepts like data persistence, modularity, and user interface creation. Additionally, it simplified the fundamental concepts of Git, particularly for students who are unfamiliar with version control. In addition to achieving its objectives, the project was an excellent learning experience for creating interactive, real-world applications.

9. References

- Oracle.(2024).Java Platform, Standard Edition 17 Documentation. Oracle Corporation. Retrieved from https://docs.oracle.com/javase/17/docs/api/
- Oracle. (2024). Introduction to JavaFX. Oracle Corporation. Retrieved from https://openjfx.io/

- Chacon, S., & Straub, B. (2014). Pro Git *(2nd ed.)*. Apress. Retrieved from https://git-scm.com/book/en/v2
- GeeksforGeeks. (2023). Understanding Version Control Systems (VCS). Retrieved from https://www.geeksforgeeks.org/version-control-systems/
- TutorialsPoint. (2023). JavaFX – Overview and Basic Controls. Retrieved from https://www.tutorialspoint.com/javafx/index.htm
- GitHub Docs. (2024). About Version Control and Git Concepts. GitHub, Inc. Retrieved from https://docs.github.com/en/get-started/using-git/about-git
- Software Pulse (John McNeil). (2019, Oct 3). Gluon *SceneBuilder getting started (part1)* [Video]. YouTube. https://youtu.be/ao1ESgIy2Ws
- Reddit user “nuclear_splines”. (2023, approximate). *How do I go about creating a simple Version Control System* [Discussion]. Reddit. Retrieved from https://www.reddit.com/r/compsci/comments/1cvrxwt/how_do_i_go_about_creating_a_simple_version/